Blunder Guard: Efficient Small Language Models for Heuristic-Guided Chess Commentary Generation

Seyed Ali Hosseininasab
Shahid Beheshti University
Tehran, Iran
seyed123ali123@gmail.com

Farima Kafi*
Shahid Beheshti University
Tehran, Iran

ABSTRACT

We present **Blunder Guard**, a locally deployable system for generating pedagogical chess commentary by integrating a domain-adapted Small Language Model (SLM) with classical engine evaluation. A 1.7B-parameter Qwen3 model is fine-tuned on unstructured classical chess literature and guided at inference time by deterministic features, evaluations, and principal variations derived from Stockfish.

The training corpus comprises over 2.8M tokens extracted from canonical chess books using a parallelized PDF extraction pipeline. Positional features are computed from Stockfish evaluations and processed through a structured feature extractor before being fed into the SLM. The model is exported to GGUF for deployment with `llama.cpp`; the fine-tuned checkpoint evaluated in this report uses Q8_0 quantization.

We evaluate seven candidate language models on 60 held-out position transitions divided equally among beginner, intermediate, and advanced strata. A local Gemma-family 12B model acts as a judge and scores correctness, instruction following, helpfulness, faithfulness, and fluency against curated references. The fine-tuned model is fluent but obtains only 2.49/10 overall, while the strongest tested hosted model obtains 7.31/10. The results show that local deployment is practical, but fluency alone does not establish chess-grounded correctness.

KEYWORDS

Chess AI, Small Language Models, Explainable AI, Natural Language Generation

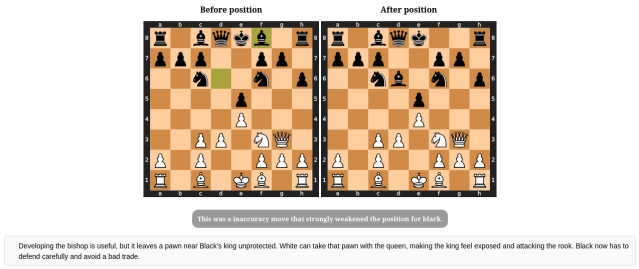


Figure 1: Blunder Guard user interface showing interactive chessboard, real-time position analysis, and generated commentary for moves before and after evaluation.

1 INTRODUCTION

The confluence of superhuman chess engines and natural language generation presents a unique opportunity for chess education: explaining *why* a position favors one side in terms accessible to human learners. Modern engines such as Stockfish [16] and AlphaZero [15] achieve tactical precision far beyond human masters, yet their output—centipawn evaluations and principal variations—remains opaque to intermediate players. Conversely, Large Language Models (LLMs) generate fluent explanations but frequently hallucinate tactical motifs and misevaluate positions, rendering them unreliable for serious instruction.

This work addresses the problem of **interpretable chess analysis**: generating commentary that is simultaneously (1) strategically accurate, grounded in objective engine evaluation; (2) pedagogically structured, following the conventions of classical chess literature; and (3) computationally efficient, enabling fully local deployment without cloud dependencies.

Existing approaches fall into two categories. Supervised methods [9] require large annotated datasets and lack scalability. LLM-based methods [10] achieve quality through scale but incur prohibitive costs and raise privacy concerns. Neither approach satisfies the needs of clubs, schools, or individual players seeking affordable, private coaching tools.

We propose a different paradigm: fine-tuning Small Language Models (SLMs) on unstructured chess literature while guiding generation with engine-verified heuristics. Canonical instructional texts encode reusable explanatory patterns without position-specific labels. By learning these patterns, an SLM can generalize to novel positions when guided by heuristic priorities derived from Stockfish, producing commentary that is both informative and pedagogically sound.

2 RELATED WORK

The intersection of artificial intelligence and chess has long served as a testbed for exploring interpretable decision-making and natural language generation. While early approaches focused primarily on playing strength [3, 15], recent research has shifted toward making strong chess engines comprehensible to human players. This evolution reflects a broader trend in explainable AI: moving from black-box superhuman performance to systems that can articulate *why* certain decisions are made [1]. In this context, chess commentary generation has emerged as a challenging benchmark, requiring models to balance strategic accuracy with pedagogical clarity. Unlike generic text generation, chess commentary demands

*Instructor: Maryam Tahmasbi

domain knowledge, precise move grounding, and the ability to convey complex positional concepts in accessible language. The closest literature spans hand-crafted game features, learned neural-engine representations, symbolic-engine control signals, and explicit strategic concepts. Table 1 summarizes the principal distinctions.

2.1 Bridging the Gap between Expert and Language Models

Kim et al. [10] address a fundamental challenge in explainable artificial intelligence: bridging the gap between expert decision-making models and large language models (LLMs). While expert models such as modern chess engines achieve superhuman performance in complex decision-making tasks, their outputs are difficult for humans to interpret. In contrast, LLMs are capable of generating fluent and natural language explanations but often suffer from hallucinations and lack domain-specific reasoning capabilities. The authors study this problem in the context of chess commentary generation, which serves as a representative task for explaining complex decisions through natural language.

To overcome these limitations, the paper proposes a framework called *Concept-guided Chess Commentary* (CCC). The key idea is to extract high-level chess concepts—such as king safety, passed pawns, mobility, and threats—from a neural chess expert model using concept-based explanation techniques. These concepts are represented as vectors learned via linear classifiers trained on positions labeled by a classical chess engine. For a given move, the framework prioritizes concepts by comparing their relevance before and after the move, identifying which concepts are most influential in explaining the decision.

The prioritized concepts are then provided as guidance to a large language model during commentary generation. This integration allows the LLM to focus on strategically important aspects of the position while avoiding incorrect or hallucinated explanations. As a result, the generated commentary is both linguistically fluent and strategically accurate.

In addition to commentary generation, the authors address the problem of evaluation. They introduce *GPT-based Chess Commentary Evaluation* (GCC-Eval), a multi-dimensional evaluation framework that assesses commentary based on relevance, completeness, clarity, and fluency. Unlike traditional similarity-based metrics such as BLEU or ROUGE, GCC-Eval incorporates expert chess knowledge and shows a significantly higher correlation with human expert judgments.

Experimental results on a standard chess commentary dataset demonstrate that the proposed CCC framework outperforms existing baselines and even rivals human-generated commentary in terms of informativeness and linguistic quality. Overall, this work highlights the effectiveness of concept-based explanations as an interface between expert models and language models, and it provides a generalizable approach for interpretable decision explanation beyond the domain of chess.

2.2 Engine-Conditioned Language Generation

The closest architectural predecessor to Blunder Guard is Lee et al. [11]. Their system combines a symbolic chess engine with a controllable language model. During training, it extracts control tags associated with properties such as move quality and the best continuation; during inference, a chess engine supplies those tags. Human judges preferred the engine-conditioned outputs to earlier baselines. Both systems therefore separate reliable chess calculation from natural-language realization. The important difference is the interface: Lee et al. learn from paired game-state/commentary data and condition generation with discrete control codes, whereas Blunder Guard fine-tunes on unpaired chess books and constructs a natural-language prompt from Stockfish multi-PV lines and deterministic before/after features. Their stronger supervised grounding helps explain why paired position-level data remains an important direction for our work.

Zang et al. [19] integrate the expert more deeply. Their automated commentator jointly trains category-specific generation models with a neural chess engine that encodes boards, predicts moves, and represents likely future positions. They evaluate five commentary categories—description, move quality, context, comparison, and planning—using both automatic and human evaluation. This differs from Blunder Guard's modular use of an external Stockfish process: joint training may yield richer continuous board representations, while the modular approach permits a small generator, an independently replaceable engine, auditable features, and local deployment without training a chess-playing network.

2.3 Learning to Generate Move-by-Move Chess Commentary

Jhamtani et al. [9] study the task of automatically generating natural language commentary for individual moves in chess games. The authors frame this problem as a grounded natural language generation task, where the generated text must accurately reflect the game state, adhere to the rules of chess, and capture the strategic and pragmatic motivations behind a move. Chess commentary generation is particularly challenging due to the need for domain knowledge, the evolving nature of the game state, and the high variability in how humans describe the same move.

A major contribution of this work is the introduction of a large-scale chess commentary dataset collected from online chess forums. The dataset contains more than 298,000 move–commentary pairs spanning over 11,000 games, making it the first dataset of this scale for move-level chess commentary generation. An analysis of the data reveals substantial diversity in commentary styles, which the authors categorize into several types, including direct move descriptions, evaluations of move quality, comparisons with alternative moves, strategic planning explanations, contextual game-level information, and general comments. This diversity highlights the importance of content selection and pragmatic reasoning in the generation process.

To address these challenges, the authors propose a neural model called *Game-Aware Commentary* (GAC). The model is trained end-to-end and incorporates multiple sources of domain-specific information extracted from the chess board state before and after a move. These features include move-related information (e.g., piece type and position), threat-related information (e.g., attacking and defending pieces), and score-based features obtained from a chess engine to estimate move quality. Feature representations are embedded in a shared continuous space, and a bidirectional LSTM encoder

Table 1: Comparison with the closest chess explanation and commentary systems.

Work	Chess grounding	Language supervision	Evaluation	Main distinction from Blunder Guard
Jhamtani et al. [9]	Hand-crafted move, threat, and engine-score features	298K paired comments	Automatic metrics and human study	Supervised feature-aware generation
Zang et al. [19]	Jointly trained neural chess engine	Paired comments by category	Automatic and human evaluation	Internal continuous engine representation
Lee et al. [11]	Symbolic-engine control tags	Paired states and comments	Perplexity and human preference	Closest architecture; learned discrete control interface
Kim et al. [10]	Prioritized concepts from an expert model	Large-LLM generation	Human and GCC-Eval	Learned concept prioritization and LLM judge
Cui et al. [5]	Pruned branching engine strategy	Prompted LLM verbalization	Executability beyond the main line	Forward-looking strategy rather than post-hoc move explanation
Blunder Guard	Stockfish multi-PV plus board deltas	Unpaired chess books; constrained prompt	References scored by local Gemma-4-12B	Small local generator and deterministic, inspectable feature interface

is used to model feature conjunctions. A selection mechanism is further employed to dynamically identify salient features during text generation, followed by an LSTM decoder that produces the commentary.

Experimental results show that the proposed GAC model outperforms several baselines, including template-based methods, nearest-neighbor approaches, and models that rely solely on raw board representations. Although automatic metrics such as BLEU yield relatively low scores due to the inherent variability of natural language commentary, human evaluation demonstrates that the generated commentaries are comparable to, and in some cases indistinguishable from, human-written commentary in terms of correctness and fluency. Overall, this work establishes a strong foundation for research on grounded and interpretable language generation in games and serves as a key reference for subsequent studies in chess commentary and explainable decision-making systems.

2.4 From Move Commentary to Strategy Verbalization

Cui et al. [5] formulate the newer task of *chess strategy verbalization*. Instead of explaining only a move that has already been played, their pipeline extracts a pruned, branching strategy tree from an engine and asks a language model to communicate a forward-looking policy. Their evaluation tests whether a human or language-model player can execute the verbalized strategy, including responses outside the engine's main line. The study finds that concrete variations matter, that concept-only descriptions are limited, and that LLM-only evaluation has weaknesses. These observations are directly relevant to Blunder Guard: our current three-PV context is richer than a single evaluation but the generated response remains post-hoc and limited to two or three sentences. Extending evaluation from reference similarity to executable multi-branch plans would test whether commentary is practically useful rather than merely plausible.

Complementary interpretability work probes what chess engines know rather than generating commentary. Pálsson and Björnsson [13] apply probing methods to a superhuman chess agent and recover human-recognizable concepts from its neural representation. Björnsson's survey [1] places such work within the broader need for chess agents that explain the rationale behind their decisions. These studies support the premise that engine representations contain explainable strategic information, but they do not by themselves solve content selection or beginner-friendly language realization. Blunder Guard addresses this last interface explicitly through board-difference heuristics and constrained prompting.

3 DATASET PREPARATION

In this section, we describe the construction of our training corpus and evaluation framework. Our approach diverges from conventional supervised learning: we employ a two-stage pipeline where (1) a Small Language Model (SLM) is first fine-tuned on pure chess literature without position-specific labels, and (2) a heuristic-enhanced evaluation system is subsequently applied to guide inference. This design choice prioritizes general strategic understanding over memorization of specific position-label pairs.

3.1 Training Corpus Composition

The training dataset comprises curated classical chess literature representing diverse pedagogical approaches and historical periods. Unlike prior work that relies on annotated game databases with move-level labels [9], our corpus consists of unstructured instructional text authored by renowned chess masters.

3.1.1 Chess Literature Corpus. We digitized five canonical chess instruction books:

- *The Game of Chess* by Siegbert Tarrasch (1935) [18]: Foundational strategic principles and systematic piece development
- *Zurich International Chess Tournament, 1953* by David Bronstein (1979) [2]: Deep annotations of candidate tournament games
- *The Life and Games of Mikhail Tal* by Mikhail Tal (1997) [17]: Attacking patterns and dynamic piece play
- *How to Reassess Your Chess* by Jeremy Silman (2010) [14]: Positional imbalances and modern strategic thinking
- *Logical Chess: Move by Move* by Irving Chernev (1957) [4]: Pedagogical explanations for intermediate players

These texts span classical principles [18], modern imbalance theory [14], tactical brilliance [17], and instructional methodology [4], providing comprehensive coverage of chess pedagogy.

3.1.2 Text Extraction and Preprocessing. Source materials existed in PDF format, necessitating robust extraction and cleaning. We developed a parallelized pipeline implemented in Python using `pypdf` for PDF parsing. Algorithm 1 describes the extraction process.

Algorithm 1 PDF Text Extraction Pipeline

Require: PDF directory D_{pdf}, Output directory D_{txt}
Ensure: Clean text files T

$F \leftarrow \textsc{ListPDFs}(D_{pdf})$ ▷ Enumerate PDF files
$T \leftarrow \emptyset$
for each file $f \in F$ **in parallel** using `ThreadPoolExecutor` **do**
  $text \leftarrow \textsc{ExtractText}(f)$ ▷ `PdfReader` page extraction
  $text \leftarrow \textsc{RemoveHyphenation}(text)$ ▷ Join broken words
  $text \leftarrow \textsc{NormalizeWhitespace}(text)$ ▷ Collapse multiple spaces/newlines
  if $\textsc{Length}(text) > \theta_{min}$ **then** ▷ Filter empty/low-quality extractions
    $t_{path} \leftarrow \textsc{Save}(text, D_{txt}, \textsc{ChangeExtension}(f, .txt))$
    $T \leftarrow T \cup \{t_{path}\}$
  end if
end for
return T

The implementation handles critical PDF artifacts: line-break hyphenation (e.g., "posi- tion" → "position") and irregular whitespace from column layouts. Parallel processing utilizes all available CPU cores for efficient batch processing of the corpus. Total extracted text: approximately 2.8 million tokens after cleaning and deduplication.

3.2 Heuristic Enhancement Module

Following model training, we employ a heuristic module to guide commentary generation during inference. This module bridges the gap between the model's general strategic knowledge and specific position evaluation, analogous to the concept extraction approach of Kim et al. [10] but implemented without requiring labeled training data.

Algorithm 2 describes the current Stockfish pipeline. For both the pre-move and post-move FEN, the engine searches to depth 17 with three principal variations. Scores and win probabilities are kept from White's point of view; the before/after delta is then sign-adjusted to the perspective of the player who made the move.

(1) Queries Stockfish [16] for multi-PV evaluations, best moves, and forced-mate status.
(2) Infers the played move by replaying legal moves from the first FEN.
(3) Compares the two boards to detect concrete structural changes.
(4) Packages engine and heuristic evidence into one prompt-ready record.

Algorithm 2 Expert-Guided Move Evaluation with Structural Feature Extraction

Require: Position FEN_{before}, Position FEN_{after}
Ensure: Structured expert representation S

$E_{before}, PV_{before}, B \leftarrow \textsc{MultiPV}(FEN_{before}, 3, 17)$
$E_{after}, PV_{after}, B' \leftarrow \textsc{MultiPV}(FEN_{after}, 3, 17)$
$\Delta \leftarrow \textsc{PerspectiveAdjustedDelta}(E_{before}, E_{after})$
$m \leftarrow \textsc{InferPlayedMove}(FEN_{before}, FEN_{after})$
$F_{white} \leftarrow \textsc{ExtractPositionFeatures}(FEN_{before}, FEN_{after}, \text{White})$
$F_{black} \leftarrow \textsc{ExtractPositionFeatures}(FEN_{before}, FEN_{after}, \text{Black})$
$Q \leftarrow \textsc{AssessMoveQuality}(\Delta, \textsc{MateStatus}(E_{after}))$
Construct structured record S with:
  evaluations $(E_{before}, E_{after}, \Delta)$
  played move m, best moves (B, B'), and principal variations
  qualitative assessment Q
  feature deltas (F_{white}, F_{black})
return S

The heuristic extracts a set of interpretable structural feature categories by comparing the board state before and after a move:

- **Evaluation Shift**: Direction and magnitude of the engine-evaluated positional change, adjusted to the perspective of the player who made the move.
- **Material Safety**: Emergence of newly hanging or undefended pieces as a consequence of the move.
- **Pawn Structure**: Introduction of structural weaknesses such as doubled, isolated, or backward pawns.
- **King Safety**: Deterioration of defensive conditions, including loss of castling rights, increased attack pressure, or weakened pawn shields.
- **Positional Control**: Changes in control of e4, e5, d4, and d5, plus delayed development of bishops and knights.
- **Tactical Context**: Best moves before and after the transition, missed-opportunity text, and forced-mate information.

These features are derived through differential analysis of consecutive positions and do not rely on explicit tactical labels. The structured record contains the two FENs, Stockfish text, numeric evaluations, player-to-move fields, best moves, the played move, move-quality text, mate status, and feature dictionaries for both sides. This explicit interface is the main change from the earlier pipeline: candidate models receive both symbolic board changes and concrete multi-PV evidence rather than a single scalar engine score.

3.3 Evaluation Data Construction

While our training corpus contains no position–commentary pairs, we construct a separate benchmark from the February 2013 Lichess standard-rated game database [12]. This benchmark is used only for assessment.

Games of at least 30 plies are grouped by the players' mean Elo: below 1200 for beginner, 1200–1799 for intermediate, and 1800 or above for advanced. Twenty manually selected transitions from each group form a balanced 60-row benchmark. For each transition, we record:

- **FEN representations**: Before and after the move
- **Engine-grounded prompt**: Stockfish lines, feature changes, move classification, and mate information
- **Move context**: SAN notation, stable row identifier, and difficulty group
- **Reference explanation**: Curated learner-facing commentary used as judging ground truth

Algorithm 3 Balanced Evaluation Benchmark Construction

Require: Clustered PGN-derived games G_c, selected move indices I_c
Ensure: Balanced reference records R

$R \leftarrow \emptyset$
for each difficulty $c \in \{beginner, intermediate, advanced\}$ **do**
  for each selected index $i \in I_c$ **do**
    $(FEN_b, FEN_a) \leftarrow \textsc{ReplayToTransition}(G_c, i)$
    $S \leftarrow \textsc{ExpertStructOutput}(FEN_b, FEN_a)$
    $p \leftarrow \textsc{BuildPrompt}(S)$
    $r \leftarrow \textsc{Record}(\textsc{StableId}(i), FEN_b, FEN_a, p, c)$
    $R \leftarrow R \cup \{r\}$
  end for
end for ▷ 20 records per group, 60 total
return R

The evaluation records are stored as tab-separated values and aggregated from the three selected-move files. Candidate generation preserves `row_id` and `type`, which permits strict one-to-one alignment with the reference. Model names are embedded in artifact filenames to prevent answer-model and judge-model metadata from drifting apart.

Table 2: Evaluation Dataset Schema

Field	Type	Description
`before_fen`	string	FEN before move
`after_fen`	string	FEN after move
`move`	string	Move in SAN format (e.g., `Nf3`)
`prompt`	string	Engine-grounded candidate prompt
`analyse`	string	Curated reference commentary
`move type`	string	Engine-derived quality class
`type`	string	Beginner, intermediate, or advanced stratum

4 METHODOLOGY

Blunder Guard is designed as a lightweight, deployable system for generating instructive chess commentary by combining classical engine evaluation with a Small Language Model (SLM). Rather than relying on large-scale end-to-end model training, the system emphasizes heuristic-guided inference, enabling strong explanatory performance while remaining feasible on consumer hardware. This section outlines the overall system architecture and data flow.

Figure 2 presents an abstract overview. The implemented API accepts two FEN strings, representing the board before and after a move. Stockfish analyzes both positions, the Python chess layer derives structural deltas, and the resulting evidence is serialized into a constrained beginner-oriented prompt. A quantized model served through `llama-server` generates the explanation, which Flask returns to the React client.

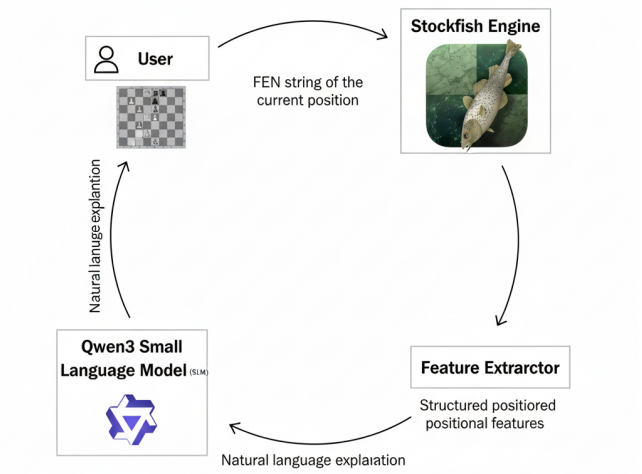


Figure 2: System architecture. The current implementation analyzes a position transition rather than a single isolated board.

4.1 System Overview

The Blunder Guard pipeline follows a modular, sequential design that integrates symbolic chess analysis with neural language generation. Given a chess position and a candidate move, the system produces a natural-language explanation assessing the move's quality and underlying ideas.

The process begins with **engine-based analysis**. Stockfish is configured with 14 threads and a 2048 MB hash table. Each position is searched to depth 17 with three principal variations. The pipeline records the leading continuation, best move, centipawn score, transformed White win probability, and mate status.

Next, a deterministic **feature extraction module** compares the two legal board states. It detects newly hanging pieces, new doubled/isolated/backward pawns, degraded castling rights or pawn shields, increased king pressure, weakened control of central squares, and delayed minor-piece development. It also infers the played move and computes a player-perspective evaluation delta used to assign a qualitative move class.

Based on these features, the system constructs a **heuristic-guided prompt**. The prompt asks for two or three short sentences, prohibits engine terminology, numeric evaluations, and quoted lines, and supplies best moves and principal variations only as hidden reasoning context. This separates evidence acquisition from learner-facing verbalization.

Finally, the structured prompt is passed to a fine-tuned **Small Language Model**, which generates human-readable commentary explaining the move in an instructive style. The SLM operates purely at inference time, with no access to engine evaluations beyond what is encoded in the prompt.

This modular architecture decouples chess analysis, feature extraction, and language generation. It also permits the same 60 prompts to be sent either to a managed local GGUF server or to a hosted OpenAI-compatible endpoint, which is essential for the comparative evaluation in Section 5.

4.2 SLM Fine-Tuning

To adapt the base language model to chess commentary, we perform targeted fine-tuning on our curated chess literature corpus. The fine-tuning procedure emphasizes efficient memory usage and stable training, enabling deployment on consumer-grade GPUs.

4.2.1 Model Architecture. The base model, **Qwen3-1.7B**, is a decoder-only transformer with 28 layers. Each layer contains multi-head attention and feed-forward submodules. Table 3 summarizes the key architectural components:

Input sequences are tokenized and divided into 256-token chunks to optimize training efficiency while preserving context for long-form chess commentary.

4.2.2 QLoRA Adaptation. We employ **QLoRA** [6] (Quantized Low-Rank Adaptation) to efficiently adapt the base model using LoRA adapters. The LoRA configuration uses rank $r = 16$ and scaling factor $\alpha = 32$, applied to selected projection matrices within the attention layers (query and value projections). A small dropout of 0.05 and no bias terms are used to improve generalization and memory efficiency.

The training dataset is prepared as a `Dataset` object and fed to a `Trainer` instance from HuggingFace Transformers. Gradient accumulation is applied to simulate larger batch sizes, and mixed-precision training (FP16) reduces GPU memory usage. After training, the LoRA adapters are merged into the base model, producing a fully adapted model that can be saved and loaded for inference.

This fine-tuning approach balances computational efficiency with performance, enabling the SLM to produce fluent, chess-informed commentary without requiring full-parameter retraining.

4.3 Quantization using `llama.cpp`

To optimize the adapted Qwen3 model for local deployment on limited hardware, we employ **quantization** using the `llama.cpp` toolkit. Quantization reduces model size and memory footprint while maintaining acceptable accuracy for inference. In our workflow, the 16-bit floating-point model is converted to 8-bit using the following command:

```
!llama.cpp/build/bin/llama-quantize \
    /kaggle/working/chess-fp16.gguf \
    /kaggle/working/chess-q8_0.gguf \
    q8_0
```

This produces a compressed model suitable for CPU and GPU inference, enabling efficient evaluation and integration into our Blunder Guard pipeline without sacrificing the quality of generated chess commentary.

4.4 Feature Extraction and Prompting

Our inference pipeline combines domain-specific chess knowledge from Stockfish with the generative capabilities of the fine-tuned LLM. The pipeline is designed to provide human-readable, pedagogical explanations of moves, suitable even for beginner players, while retaining the analytical rigor of engine evaluations.

4.4.1 Positional Feature Extraction. The first step combines Stockfish output with board-difference features computed using the `python-chess` library [7]. The current implementation captures:

- **Central control**: Control over key central squares.
- **Material safety**: Newly attacked pieces that have no defender.
- **King safety**: Lost castling rights, added direct pressure, or a reduced pawn shield.
- **Pawn structure**: Newly doubled, isolated, or backward pawns.
- **Development and tactics**: Minor pieces remaining on their starting squares, best continuations, missed opportunities, and forced mate.

The numerical and line-based evidence is present in the prompt as context, but the output contract prevents the model from repeating it to a beginner. This distinction is important: the generator is exposed to engine evidence even though the final commentary should contain only its pedagogical interpretation.

4.4.2 Inference Approaches. We implement two complementary methods for generating move explanations:

Single Inference Method. In this approach, the LLM directly receives the positional features extracted from Stockfish and generates a natural language explanation in one step. This method is efficient and produces concise commentary, making it suitable for fast, real-time feedback or integration into a user interface where simplicity is preferred. The main steps are:

Table 3: Qwen3-1.7B Model Architecture Overview

Component	Details
Model type	Decoder-only Transformer (Qwen3ForCausalLM)
Embedding layer	Token embedding: 151,936 vocab size, hidden size 2048
Number of layers	28 (Qwen3DecoderLayer)
Attention	Qwen3Attention per layer: - Query projection: 2048→2048 - Key projection: 2048→1024 - Value projection: 2048→1024 - Output projection: 2048→2048 - Layer norms: q_norm, k_norm (RMSNorm, 128-dim)
Feed-forward network (MLP)	Gate: 2048→6144 Up: 2048→6144 Down: 6144→2048 Activation: SiLU
Layer normalization	Input and post-attention RMSNorm (2048-dim)
Positional embedding	Rotary embedding
Language modeling head	Linear: 2048→151,936

Algorithm 4 Single Inference Method (Abstract)

Require: Stockfish evaluation of current position
Ensure: Beginner-friendly move explanation
1: Extract position features from engine output
2: Construct a natural language prompt using these features
3: Feed the prompt into the LLM
4: Receive and return textual explanation

Chain Inference Method. The chain inference method produces more nuanced explanations by performing multi-step reasoning. First, the LLM evaluates the positions *before* and *after* a move independently, generating structured descriptions for each. These intermediate outputs, together with Stockfish analysis, are then combined in a second prompt to produce a final explanation that clearly articulates why the move improved or weakened the position. This method allows the model to perform its own comparative reasoning in addition to leveraging engine insights, resulting in more detailed and instructive commentary. The workflow can be summarized as:

Algorithm 5 Chain Inference Method (Abstract)

Require: Stockfish analysis of **before** and **after** positions
Ensure: Detailed move explanation
1: Extract positional features for both before and after positions
2: Construct separate LLM prompts for each position
3: Feed prompts into LLM to obtain intermediate textual summaries
4: Compare before/after summaries with Stockfish evaluation
5: Construct a final LLM prompt incorporating all intermediate information
6: Feed final prompt into LLM
7: Receive and return the final textual explanation

4.4.3 Discussion and Use Cases. The single inference method is ideal for scenarios requiring rapid evaluation or lightweight deployment, such as mobile apps or web interfaces. The chain inference method, although computationally heavier, is better suited for detailed post-game analysis, teaching tools, and generating insights that require reasoning over position changes. Together, these two approaches provide a flexible framework that balances speed, clarity, and analytical depth, ensuring the system can serve both casual players and serious learners.

4.5 Deployment and Software Architecture

The Blunder Guard system is implemented as a full-stack web application, combining a Python-based backend for chess analysis with a modern JavaScript frontend for interactive visualization. Unlike many academic or research-oriented projects, Blunder Guard is fully deployable, allowing users to run it locally or on any server, making it accessible across a wide range of devices from desktops to laptops and even mobile browsers. The backend handles model inference, Stockfish evaluation, and API endpoints, while the frontend provides a responsive interface for users to explore game commentary in real time.

4.5.1 Backend. The backend is implemented in the `app` directory using Python and Flask. It is responsible for:

- Serving API endpoints for LLM inference and Stockfish evaluation
- Validating before/after FEN inputs supplied by the frontend
- Integrating quantized Qwen3 LLM models using `llama.cpp` for efficient local inference on consumer-grade hardware
- Returning a JSON object containing the generated commentary

4.5.2 Frontend. The frontend is implemented in the `frontend` directory using React and Bun. It provides:

- Interactive chessboard display with move highlighting

- Commentary requests to the single-step and chain endpoints
- Presentation of the generated move explanation
- Adaptive design for desktop, tablet, and mobile devices, ensuring consistent user experience across platforms
- A production build served directly by Flask

This architecture ensures smooth communication between the backend LLM inference engine and the frontend visualization, enabling an accessible, deployable, and fully operational chess coaching system that can be run on virtually any device without extensive setup. The software interface is shown in Figure 3.

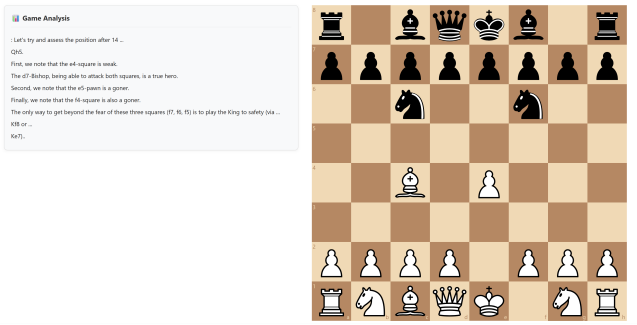


Figure 3: Blunder Guard user interface showing game analysis panel (left) and interactive chessboard (right).

5 EVALUATION

Standard natural language generation metrics such as BLEU, ROUGE, and BERTScore are commonly used to evaluate text generation systems. However, these metrics are not suitable for our setting because **our training corpus contains no paired position–commentary data**. Unlike supervised approaches [9, 10], where models learn to reproduce human commentary for specific positions, Blunder Guard generates *de novo* explanations based on general chess knowledge extracted from literature.

A generated explanation may be linguistically fluent yet strategically incorrect. For example, a commentary stating "White gains a decisive advantage by activating the rook" may read well but be objectively false if the position is evaluated as equal by Stockfish. Conversely, a factually correct explanation phrased in a non-canonical style may appear stylistically unusual while being practically more useful.

5.1 Candidate Models and Protocol

We evaluate seven answer models: the fine-tuned Qwen3 model (`chess-q8_0`), Llama-3.2-3B-Instruct, Gemma-3-12B-Instruct, Gemini-3.5-Flash, GPT-5.5, Grok-4.3, and Qwen3.7-Plus. Each model receives the same 60 engine-grounded prompts. Local GGUF models are launched through a managed `llama-server`; hosted models are accessed through an OpenAI-compatible API. The answer-model identity is stored both in each row and in the output filename.

We then compare every candidate response with its aligned reference using a local **Gemma 4 12B judge** [8]. This follows the LLM-as-a-judge paradigm [20], adapted to reference-grounded chess evaluation. Reasoning is disabled, temperature is zero, and the judge is required to return schema-constrained JSON. It assigns independent integer scores from 0 to 10 for:

- **Correctness**: Agreement with the central chess claims in the reference.
- **Instruction following**: Relevance as a learner-facing chess explanation.
- **Helpfulness**: Communication of the reference lesson, cause, or consequence.
- **Faithfulness**: Avoidance of claims unsupported by the reference.
- **Fluency**: Grammar and clarity, scored independently of factual accuracy.

The application calculates a weighted overall score:

$$O = 0.35C + 0.25F + 0.20H + 0.10I + 0.10L, \tag{1}$$

where C, F, H, I, and L denote correctness, faithfulness, helpfulness, instruction following, and fluency. Correctness and faithfulness therefore account for 60% of the final score. Candidate/reference rows are aligned by difficulty type and stable row ID. All 420 judgments parsed successfully; no Qwen-judge output is included in the reported results.

5.2 Results

Table 4 reports means over all 60 positions for each answer model. GPT-5.5 ranks first with 7.31 overall, followed by Qwen3.7-Plus (6.50), Gemini-3.5-Flash (6.46), and Grok-4.3 (5.44). The three local models score below 3.0. The fine-tuned `chess-q8_0` model obtains 2.49, below the untuned Llama-3.2-3B (2.86) and Gemma-3-12B (2.95) baselines.

Figure 4 makes the separation between surface quality and grounding visible. Most models receive near-perfect fluency and instruction-following scores, while correctness and faithfulness determine the ranking. Figure 5 shows that advanced positions are generally harder: for example, Gemini falls from 7.10 on beginner positions to 5.19 on advanced positions, and GPT-5.5 falls from 7.36 to 6.72. Qwen3.7-Plus is comparatively stable across the three groups.

These results do not support the earlier qualitative claim that the fine-tuned small model is reliably strategically accurate. They instead show that its text can look plausible while disagreeing with the reference lesson or introducing unsupported chess claims. Fine-tuning on unpaired books appears to improve domain style less reliably than explicit position-grounded supervision would improve factual alignment.

6 DISCUSSION

Blunder Guard demonstrates a complete lightweight pipeline for combining classical engine evidence with language generation, but the quantitative evaluation also identifies a clear quality limit. The fine-tuned model follows the intended local-deployment design, yet it does not reliably translate the supplied evidence into reference-aligned chess explanations.

6.1 Strengths of the Approach

Our methodology has several notable advantages:

Table 4: Gemma-4-12B judge means over 60 positions per answer model (0–10).

Answer model	Correct.	Instr.	Helpful	Faithful	Fluent	Overall
GPT-5.5	6.28	9.85	7.57	6.43	10.00	**7.31**
Qwen3.7-Plus	5.30	10.00	6.63	5.27	10.00	6.50
Gemini-3.5-Flash	5.15	9.97	6.70	5.27	10.00	6.46
Grok-4.3	3.92	9.90	5.27	4.08	10.00	5.44
Gemma-3-12B-Instruct Q4	0.88	10.00	2.03	0.92	10.00	2.95
Llama-3.2-3B-Instruct Q4	0.83	9.40	1.95	0.93	10.00	2.86
Fine-tuned chess Q8	0.83	7.22	1.82	0.88	8.97	2.49

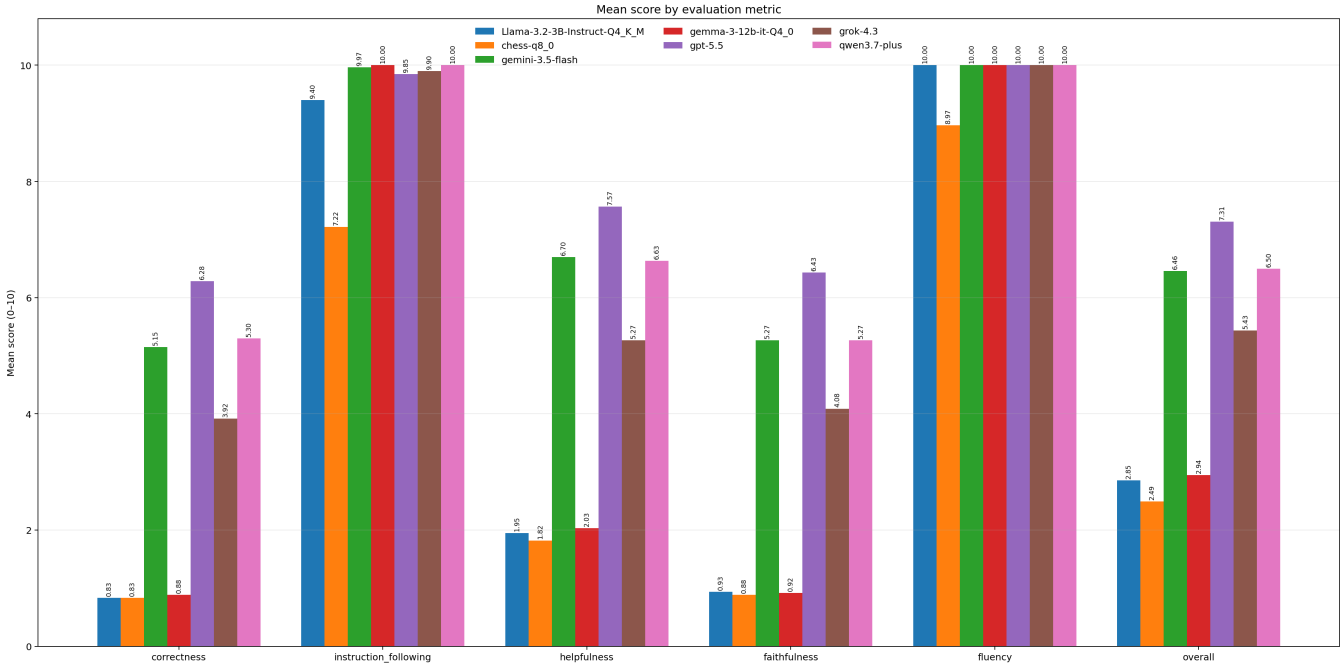


Figure 4: Mean metric scores for seven answer models, evaluated only by the Gemma-4-12B judge.

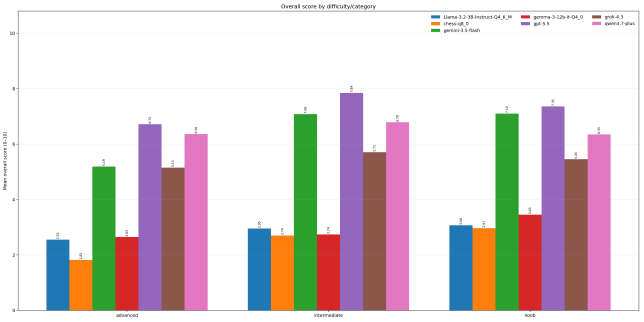


Figure 5: Mean weighted overall score by benchmark difficulty. Each model has 20 positions per group.

- **Efficiency and Deployability:** The system can run on consumer-grade hardware thanks to LoRA fine-tuning and quantization with `llama.cpp`, making it more accessible than larger, end-to-end models.
- **Inspectable grounding:** Stockfish lines, scores, best moves, mate status, and deterministic board features are separated from generation and can be audited independently.
- **Flexibility:** The dual inference methods—single-step and chain inference—allow trade-offs between speed and depth of explanation.

6.2 Limitations

Despite these strengths, several limitations should be acknowledged:

- **Judge dependence:** Scores come from one local Gemma-4-12B judge. Although deterministic prompts, strict JSON, references, and independent metrics improve reproducibility, LLM judgments can contain systematic bias and should be calibrated against rated human evaluators.

- **Reference dependence:** Each candidate is compared with one curated explanation. A different but valid strategic explanation can receive too little credit, while errors in a reference propagate into every candidate score.
- **Small benchmark:** The benchmark contains only 60 positions (20 per group), so small differences between similarly ranked models should not be treated as definitive.
- **Weak fine-tuned result:** The fine-tuned model's 2.49 overall score and sub-1.0 correctness/faithfulness means show that unpaired book fine-tuning plus prompt grounding is insufficient for reliable commentary.
- **Limited Contextual Depth:** The model operates primarily on immediate positional features and may miss long-term strategic plans that extend beyond the analyzed move.
- **Chess Knowledge Dependency:** The system relies on Stockfish for objective evaluation. In positions where engine assessments are ambiguous or non-intuitive, the commentary may lack subtlety appreciated by human players.

6.3 Future Directions

There are several avenues to improve and extend Blunder Guard:

- **Supervised Data Integration:** Incorporating labeled position-commentary pairs could improve the model's ability to generate accurate and instructive commentary. Such datasets could be created either by expert annotators or by leveraging LLMs to generate synthetic training data.
- **Hardware Optimization:** Optimizing the model to run efficiently on NPUs, mobile devices, or local systems would enhance accessibility and real-time inference capabilities.
- **Expanded Knowledge Base:** Integrating additional chess literature and opening/middle/endgame manuals could improve the breadth and depth of the model's strategic understanding.
- **Robust Evaluation on Seen vs. Unseen Positions:** Splitting chess positions from the training books and evaluating on unseen positions would allow a more systematic assessment of the model's generalization capabilities.
- **Sequence-Level Analysis:** Extending the chain inference method to multi-move sequences could enable the generation of commentary for positional trends and long-term planning.
- **Human calibration of the judge:** A blinded expert study should measure correlation between Gemma-4-12B scores and rated-player judgments, including inter-annotator agreement.
- **Ablation studies:** Stockfish lines, structural features, book fine-tuning, and chain prompting should be removed one at a time to determine which components improve correctness rather than only fluency.

Overall, the engineering architecture is promising for accessible chess explanation, but the present small-model checkpoint should be treated as a prototype rather than a reliable coach. The evaluation indicates that future work must prioritize factual grounding and position-specific supervision over stylistic fluency.

7 CONCLUSION

We presented **Blunder Guard**, a lightweight, deployable system for chess commentary that combines depth-17 multi-PV Stockfish analysis, deterministic board-difference features, constrained prompting, and quantized local inference. The application and evaluation tools run through OpenAI-compatible interfaces, allowing local and hosted answer models to be compared under one artifact-tracked pipeline.

On 60 curated positions, the Gemma-4-12B judge ranks GPT-5.5 highest at 7.31/10, while the fine-tuned local chess model scores 2.49/10. Its relatively high fluency does not compensate for low correctness and faithfulness. The principal conclusion is therefore methodological rather than a claim of solved commentary generation: engine grounding is necessary, but the current prompt and unpaired book fine-tuning do not ensure that a small model uses that evidence correctly. Position-grounded training, larger benchmarks, ablations, and human calibration are required before deployment as a dependable instructional system.

ACKNOWLEDGMENTS

This work was supported by publicly available chess literature, the Stockfish engine, Qwen3 and open-source software such as `llama.cpp`, which enabled local inference and model quantization.

REFERENCES

[1] Yngvi Björnsson. 2024. Chess and Explainable AI. *ICGA Journal* 46, 2 (2024). https://doi.org/10.3233/ICG-240256

[2] David Bronstein. 1979. *Zurich International Chess Tournament, 1953* (revised ed.). Dover Publications, New York, NY. Annotated games from the landmark candidates tournament.

[3] Murray Campbell, A. Joseph Jr. Hoane, and Feng-hsiung Hsu. 2002. Deep Blue. *Artificial Intelligence* 134, 1–2 (2002), 57–83. https://doi.org/10.1016/S0004-3702(01)00129-1 Landmark defeat of world chess champion Garry Kasparov.

[4] Irving Chernev. 1957. *Logical Chess: Move by Move* (1st ed.). Simon & Schuster, New York, NY. Detailed annotations explaining the reasoning behind every move.

[5] Langyuan Cui, Chun Kai Ling, and Hwee Tou Ng. 2026. Communicating Chess Strategies in Natural Language. *arXiv preprint arXiv:2607.11486* (2026). https://doi.org/10.48550/arXiv.2607.11486 arXiv:2607.11486 [cs.CL]

[6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. *arXiv preprint arXiv:2305.14314* (2023). https://arxiv.org/abs/2305.14314 Parameter-efficient fine-tuning method used in this work.

[7] Niklas Fiekas. 2024. python-chess: A Chess Library with Move Generation and Validation. https://python-chess.readthedocs.io/ Python library for chess data processing.

[8] Google DeepMind. 2026. Gemma 4 Model Card. https://ai.google.dev/gemma/docs/core/model_card_4 The 12B instruction model is used as the local evaluation judge.

[9] Harsh Jhamtani, Varun Gangal, Eduard Hovy, Graham Neubig, and Taylor Berg-Kirkpatrick. 2018. Learning to Generate Move-by-Move Commentary for Chess Games from Large-Scale Social Forum Data. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Long Papers)*. Association for Computational Linguistics, Melbourne, Australia, 1661–1671.

[10] Jaechang Kim, Jinmin Goh, Inseok Hwang, Jaewoong Cho, and Jungseul Ok. 2025. Bridging the Gap between Expert and Language Models: Concept-guided Chess Commentary Generation and Evaluation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, Albuquerque, New Mexico, 9497–9516. https://doi.org/10.18653/v1/2025.naacl-long.481

[11] Andrew Lee, David Wu, Emily Dinan, and Mike Lewis. 2022. Improving Chess Commentaries by Combining Language Models with Symbolic Reasoning Engines. *arXiv preprint arXiv:2212.08195* (2022). https://doi.org/10.48550/arXiv.2212.08195 arXiv:2212.08195 [cs.CL]

[12] Lichess.org. 2013. Lichess Open Database: Standard Rated Games. https://database.lichess.org/standard/ February 2013 PGN archive used to construct the evaluation strata.

[13] Aðalsteinn Pálsson and Yngvi Björnsson. 2023. Unveiling Concepts Learned by a World-Class Chess-Playing Agent. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, 4864–4872. https://doi.org/10.24963/ijcai.2023/541

[14] Jeremy Silman. 2010. *How to Reassess Your Chess: Chess Mastery Through Chess Imbalances* (4th ed.). Siles Press, Los Angeles, CA. Comprehensive guide to positional play and strategic thinking.

[15] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. 2018. A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play. *Science* 362, 6419 (2018), 1140–1144. https://doi.org/10.1126/science.aar6404

[16] Stockfish Team. 2024. Stockfish: A Free and Strong Chess Engine. https://stockfishchess.org/ Open-source chess engine used for position evaluation.

[17] Mikhail Tal. 1997. *The Life and Games of Mikhail Tal* (revised ed.). Everyman Chess, London, UK. Autobiography and annotated games of the eighth World Chess Champion.

[18] Siegbert Tarrasch. 1935. *The Game of Chess* (1st ed.). David McKay Company, Philadelphia, PA. Classic instructional text covering fundamental strategic principles.

[19] Hongyu Zang, Zhiwei Yu, and Xiaojun Wan. 2019. Automated Chess Commentator Powered by Neural Chess Engine. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Florence, Italy, 5952–5961. https://doi.org/10.18653/v1/P19-1597

[20] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685* (2023). arXiv:2306.05685 [cs.CL]